Neural Net Training

CHEAT SHEET NumPy np & PyTorch torch, F = torch.nn.functional

1 Activations › 2 Loss functions › 3 Optimizers › 4 Metrics

Activation functions

	FORMULA · DERIVATIVE	NUMPY	PYTORCH
Sigmoid	$\sigma(x) = 1 / (1 + e^{-x})$ $\sigma' = \sigma(1 - \sigma)$	`1/(1+np.exp(-x))`	`torch.sigmoid(x)`
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$ $\tanh' = 1 - \tanh^2$	`np.tanh(x)`	`torch.tanh(x)`
ReLU	$\max(0, x)$ $\text{ReLU}' = 1$ if $x > 0$ else 0	`np.maximum(0, x)`	`F.relu(x)`
Leaky ReLU	x if x > 0 else αx, $\alpha \approx 0.01$	`np.where(x>0, x, a*x)`	`F.leaky_relu(x)`
GELU	$x \cdot \Phi(x) \approx \frac{1}{2}x(1 + \tanh[\sqrt{(2/\pi)}(x + 0.0447x^3)])$	`.5*x*(1+np.tanh(.7979 *(x+.0447*x**3)))`	`F.gelu(x)`
SiLU / Swish	$x \cdot \sigma(x)$, used in Llama	`x/(1+np.exp(-x))`	`F.silu(x)`
Softmax	$e^{x_i} / \Sigma_j e^{x_j}$ subtract max(x) for stability	`e = np.exp(x-x.max())` `e / e.sum()`	`F.softmax(x, dim=-1)`

Loss functions

	FORMULA (MEAN OVER N)	NUMPY	PYTORCH
MSE (L2)	$1/n \, \Sigma \, (y - \hat{y})^2$	`np.mean((y-p)**2)`	`F.mse_loss(p, y)`
MAE (L1)	$1/n \, \Sigma \, \lvert y - \hat{y} \rvert$, robust to outliers	`np.mean(np.abs(y-p))`	`F.l1_loss(p, y)`
Huber	$\frac{1}{2}e^2$ if $\lvert e \rvert \le \delta$ else $\delta(\lvert e \rvert - \frac{1}{2}\delta)$	`a = np.abs(y-p)` `np.mean(np.where(a<=d, .5*a**2, d*(a-.5*d)))`	`F.huber_loss(p, y)`
Binary CE	$-[y \log p + (1 - y) \log(1 - p)]$ $p = \sigma(z)$; $\partial L / \partial z = p - y$	`-np.mean(y*np.log(p) +(1-y)*np.log(1-p))`	`F.binary_cross_ entropy_with_ logits(z, y)`
Cross-entropy	$-\log \text{softmax}(z)_y = -z_y + \log \Sigma_j e^{z_j}$ $\partial L / \partial z = \text{softmax}(z) - \text{onehot}(y)$	`-np.log(P[range(n),y] ).mean()`	`F.cross_entropy(z, y)`
KL div.	$\Sigma \, p \log(p / q)$, distillation	`np.sum(p*np.log(p/q))`	`F.kl_div(q.log(), p, reduction= "batchmean")`

*PyTorch's **cross_entropy** and **BCE...with_logits** take raw **logits** (no softmax / sigmoid first): they're fused and numerically stable.*

TRAINING CHECKLIST

Normalize inputs	zero mean, unit variance (train stats only)
Check initial loss	≈ ln K for K balanced classes; NaN → lower lr, clip grads
Overfit one batch	loss → 0, else there's a bug
train() / eval()	toggle dropout & BatchNorm
Train ↓ val ↑	overfit: dropout, decay, more data, early stop

AUTOGRAD & GRADIENTS

Chain rule	$\partial L / \partial x = \partial L / \partial y \cdot \partial y / \partial x$, layer by layer	`loss.backward(); w.grad`
Grad check	$f'(x) \approx (f(x + h) - f(x - h)) / 2h$, $h \approx 1e\text{-}5$	`torch.autograd.gradcheck(f, x)`
No grad	stop tracking (eval, frozen parts)	`torch.no_grad(), x.detach()`

Optimizers

	UPDATE RULE (g = $\nabla_\theta L$, η = learning rate)	TORCH.OPTIM
SGD	$\theta \leftarrow \theta - \eta \cdot g$	`SGD(params, lr=0.1)`
Momentum	$v \leftarrow \mu v + g$; $\theta \leftarrow \theta - \eta \cdot v$	`SGD(..., momentum=0.9)`
RMSProp	$s \leftarrow \rho s + (1 - \rho) g^2$; $\theta \leftarrow \theta - \eta \cdot g / (\sqrt{s} + \varepsilon)$	`RMSprop(..., alpha=0.99)`
Adam	$m \leftarrow \beta_1 m + (1 - \beta_1) g$; $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$ $\hat{m} = m / (1 - \beta_1^t)$, $\hat{v} = v / (1 - \beta_2^t)$; $\theta \leftarrow \theta - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$	`Adam(..., lr=1e-3, betas=(0.9, 0.999))`
AdamW	Adam + decoupled decay $\theta \leftarrow \theta - \eta \lambda \theta$	`AdamW(..., weight_decay=0.01)`

```python
def adamw_step(w, g, m, v, t, lr=1-3, b1=0.9, b2=0.999,
               eps=1-8, wd=0.01):            # NumPy, t starts at 1
    m = b1 * m + (1 - b1) * g                 # 1st moment (mean)
    v = b2 * v + (1 - b2) * g ** 2            # 2nd moment (variance)
    m_hat, v_hat = m / (1 - b1 ** t), v / (1 - b2 ** t)  # bias fix
    w = w - lr * (m_hat / (np.sqrt(v_hat) + eps) + wd * w)
    return w, m, v
```

LEARNING-RATE SCHEDULES (TORCH.OPTIM.LR_SCHEDULER)

StepLR	$\eta \cdot \gamma^{\lfloor \text{epoch} / s \rfloor}$	`StepLR(opt, 30, gamma=0.1)`
Cosine	$\eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi t / T))$	`CosineAnnealingLR(opt, T_max=T)`
Warmup	linear ramp ~0 → η over first steps	`LinearLR(opt, 0.01, total_iters=500)`
OneCycle	warm up to max_lr, then anneal	`OneCycleLR(opt, max_lr, total_steps=N)`
On plateau	× factor when the val metric stalls	`ReduceLROnPlateau(opt, patience=5)`

Metrics

	FORMULA	NUMPY	PYTORCH
Accuracy	(TP + TN) / N	`np.mean(pred == y)`	`(pred==y).float() .mean()`

Precision	TP / (TP + FP), predicted + that are right	`p1, y1 = pred==1, y==1`
		`tp = (p1 & y1).sum()`
		`fp = (p1 & ~y1).sum()`
		`fn = (~p1 & y1).sum()`
		`P, R = tp/(tp+fp), tp/(tp+fn)`
		`f1 = 2*P*R / (P + R)`
			`torchmetrics. F1Score(task= "multiclass", num_classes=K)`

Recall	TP / (TP + FN), real + that are found		
F1	2PR / (P + R), for imbalanced data		
Confusion	C[i, j] = #(true i, predicted j)	`C=np.zeros((K,K),int)` `np.add.at(C,(y,pred),1)`	`torchmetrics. ConfusionMatrix`
ROC-AUC	area under the TPR-vs-FPR curve	`roc_auc_score(y, s)` (sklearn)	`torchmetrics. AUROC`
R^2	$1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$	`1 - ((y-p)**2).sum() / ((y-y.mean())**2) .sum()`	`torchmetrics. R2Score`
Perplexity	exp(mean cross-entropy)	`np.exp(ce_loss)`	`torch.exp(loss)`

Init, norm & regularization

He / Kaiming	$W \sim N(0, 2 / n_{in})$, for ReLU nets	`nn.init.kaiming_normal_(w)`
Xavier	$\text{Var} = 2 / (n_{in} + n_{out})$, tanh / sigmoid	`nn.init.xavier_uniform_(w)`
BatchNorm	$\gamma(x - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)} + \beta$, stats over batch	`nn.BatchNorm1d(n)`
LayerNorm	same, stats over features of each sample	`nn.LayerNorm(n)`
Dropout	zero with prob p, scale rest by 1 / (1 − p)	`nn.Dropout(p=0.1)`
Weight decay	$L + \lambda \lvert w \rvert^2$, or $\theta \leftarrow \theta - \eta \lambda \theta$ (decoupled)	`AdamW(weight_decay=0.01)`

MLP from scratch (NumPy)

Backprop is the chain rule for a dense layer **Z = XW + b**: **dW = X^T·dZ**, **db = Σ dZ**, **dX = dZ·W^T**.

```python
import numpy as np
def softmax(z):
    e = np.exp(z - z.max(1, keepdims=True))     # stable
    return e / e.sum(1, keepdims=True)

rng = np.random.default_rng(0)                  # X: [N, D], y: [N]
W1 = rng.normal(0, np.sqrt(2 / D), (D, H)); b1 = np.zeros(H)
W2 = rng.normal(0, np.sqrt(2 / H), (H, K)); b2 = np.zeros(K)
for epoch in range(200):
    Z1 = X @ W1 + b1; A1 = np.maximum(0, Z1)    # ReLU
    P = softmax(A1 @ W2 + b2)
    loss = -np.log(P[np.arange(N), y]).mean()
    dZ2 = P.copy(); dZ2[np.arange(N), y] -= 1   # dL/dz
    dZ2 /= N                                    # softmax-onehot
    dW2, db2 = A1.T @ dZ2, dZ2.sum(0)
    dZ1 = (dZ2 @ W2.T) * (Z1 > 0)               # ReLU'
    dW1, db1 = X.T @ dZ1, dZ1.sum(0)
    for p, g in [(W1, dW1), (b1, db1), (W2, dW2), (b2, db2)]:
        p -= lr * g                             # SGD step
```

Same in PyTorch

```python
model = nn.Sequential(nn.Linear(D, H), nn.ReLU(),
                      nn.Dropout(0.1), nn.Linear(H, K)).to(dev)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.AdamW(model.parameters(), lr=1-3)
sched = torch.optim.lr_scheduler.OneCycleLR(
    opt, max_lr=1-3, total_steps=E * len(train_dl))
for epoch in range(E):
    model.train()                           # dropout / BN on
    for xb, yb in train_dl:
        xb, yb = xb.to(dev), yb.to(dev)
        loss = loss_fn(model(xb), yb)       # logits in
        opt.zero_grad()                     # grads accumulate!
        loss.backward()                     # autograd = backprop
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step(); sched.step()
    model.eval(); correct = 0
    with torch.no_grad():
        for xb, yb in val_dl:
            pred = model(xb.to(dev)).argmax(1)
            correct += (pred == yb.to(dev)).sum().item()
    acc = correct / len(val_dl.dataset)
```


github.com/arnabdeypolimi/pytorch-cheatsheets / CC BY 4.0 / v1.0 · 10/2026